💡 Database Indexing – General Guidelines for Any Django Project

---

1️⃣ Identify fields for indexing

প্রথমেই বুঝতে হবে কোন fields index করা দরকার।

✅ **Ideal candidates for indexing:**

1. Frequently **queried fields** (filter, search)

   ```
   Product.objects.filter(category='Electronics')
   ```

2. Frequently **ordered fields**

   ```
   Product.objects.all().order_by('-created_at')
   ```

3. Fields used in **foreign key relationships**
   - `ForeignKey`, `OneToOneField`, `ManyToManyField`
4. Unique fields (already indexed by default in DB)
   - e.g., `username`, `email`

❌ Avoid indexing **large text fields** unnecessarily (like `TextField`) → slows down inserts/updates.

2️⃣ Add indexes in Django models

```
from django.db import models

class Product(models.Model):
    name = models.CharField(max_length=100)
    category = models.CharField(max_length=50)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        indexes = [
            models.Index(fields=['name']),           # frequent search
field
            models.Index(fields=['category']),       # filter field
            models.Index(fields=['-created_at']),    # ordering field
```

```
        ]
        ordering = ['created_at']  # default order
```

Tips:

- Composite Index: `models.Index(fields=['category', 'price'])` → multiple filter field optimization
- Unique Constraint → automatically creates index:

```
models.UniqueConstraint(fields=['email'], name='unique_email_idx')
```

3️⃣ Optimize Querysets

Index alone isn't enough। Query optimization দরকার।

- **Select only necessary fields**

```
Product.objects.only('id', 'name', 'price')
```

- **Use `select_related` for ForeignKeys**

```
Product.objects.select_related('category').all()
```

- **Use `prefetch_related` for ManyToMany**

```
Product.objects.prefetch_related('tags').all()
```

- **Always order/filter on indexed fields**

```
Product.objects.filter(category='Electronics').order_by('-created_at')
```

4️⃣ Use DRF / Django Filter & Ordering (if API project)

```
from rest_framework import viewsets, filters
from django_filters.rest_framework import DjangoFilterBackend

class ProductViewSet(viewsets.ModelViewSet):
    queryset = Product.objects.select_related('category')
    serializer_class = ProductSerializer
```

```
    filter_backends = [DjangoFilterBackend, filters.OrderingFilter,
filters.SearchFilter]
    filterset_fields = ['category', 'price']     # indexed fields only
    search_fields = ['name']                     # indexed search field
    ordering_fields = ['created_at', 'price']    # indexed fields
    ordering = ['created_at']
```

5️⃣ Add Pagination

Huge tables → avoid loading all data at once.

```
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS':
'rest_framework.pagination.LimitOffsetPagination',
    'PAGE_SIZE': 20
}
```

6️⃣ Monitor & Adjust

1. Use `EXPLAIN` in DB or Django Debug Toolbar → check which queries are slow
2. Add index on fields that appear in **WHERE / ORDER BY** clauses in slow queries
3. Avoid over-indexing → too many indexes slows down inserts/updates

---

7️⃣ Migration Commands

After adding indexes:

```
python manage.py makemigrations
python manage.py migrate
```

> Django will create database-level indexes automatically.

8️⃣ Checklist for Any Project

- Identify frequently filtered/searched fields
- Identify frequently ordered fields
- Add `models.Index` in `Meta.indexes`

- Optimize queryset with `select_related` / `only` / `prefetch_related`
- DRF filter/order only on indexed fields
- Add pagination if table is huge
- Monitor slow queries → add/remove indexes as needed

🎯 **Pro Tip:**
যেকোনো নতুন project start করার আগে **DB indexing blueprint** তৈরি করো। যেকোন field যা future API / query এ use হবে, সেই field আগে থেকে index করা থাকলে production-ready performance ready থাকবে।

1️⃣ Model এ Indexing করা

DRF backend Django ORM ব্যবহার করে। তাই প্রথমেই model indexing দিতে হবে।

```
# products/models.py
from django.db import models

class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    category = models.CharField(max_length=50)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        indexes = [
            models.Index(fields=['name']),              # name-এ index
            models.Index(fields=['category']),          # category-এ index
            models.Index(fields=['-created_at']),      # latest first index
        ]
        ordering = ['created_at']  # default ordering first to last

    def __str__(self):
        return self.name
```

✅ **Important Notes:**

- `models.Index(fields=[...])` → database-level indexing দেয়।
- `ordering = ['created_at']` → DRF/ListAPIView এ default first-to-last order দেয়।

2️⃣ Serializer তৈরি করা

```python
# products/serializers.py
from rest_framework import serializers
from .models import Product

class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = '__all__'
```

3️⃣ DRF Viewset বা APIView তৈরি করা

DRF তে query optimization করতে হলে indexing field-এ **ordering, filter** use করতে হবে।

```python
# products/views.py
from rest_framework import viewsets, filters
from .models import Product
from .serializers import ProductSerializer

class ProductViewSet(viewsets.ModelViewSet):
    queryset = Product.objects.all()  # indexed field-এ order_by করা যাবে
    serializer_class = ProductSerializer

    # Filter & Ordering
    filter_backends = [filters.OrderingFilter, filters.SearchFilter]
    search_fields = ['name', 'category']   # indexed field faster search
    ordering_fields = ['created_at', 'price']  # only indexed fields
recommended
    ordering = ['created_at']  # default first to last
```

4️⃣ DRF URLs & Router Setup

```python
# products/urls.py
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import ProductViewSet

router = DefaultRouter()
router.register(r'products', ProductViewSet, basename='products')

urlpatterns = [
    path('', include(router.urls)),
]
```

5️⃣ Pagination & Performance Boost

DRF-এ যদি table বড় হয়, **pagination** দিয়ে memory load কমাতে হবে:

```
# settings.py
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS':
'rest_framework.pagination.LimitOffsetPagination',
    'PAGE_SIZE': 20,
    'DEFAULT_FILTER_BACKENDS': [
        'django_filters.rest_framework.DjangoFilterBackend',
        'rest_framework.filters.OrderingFilter',
        'rest_framework.filters.SearchFilter',
    ]
}
```

- `LimitOffsetPagination` → efficient for large datasets
- Always **order/filter on indexed field** → DB query fast হবে

6️⃣ Example Query: First to Last API Call

API URL: `/api/products/?ordering=created_at&search=electronics&limit=10&offset=0`

- `ordering=created_at` → first-to-last order
- `search=electronics` → indexed search field
- `limit=10&offset=0` → pagination

7️⃣ Advanced Optimization Tips

1. **Select only necessary fields** (avoid `select *`)

```
queryset = Product.objects.only('id','name','price','created_at')
```

2. **Use `select_related` or `prefetch_related`** for related models:

```
queryset = Product.objects.select_related('category').all()
```

3. **Database-level Indexing**:
   - Unique fields → `models.UniqueConstraint`

- Composite indexing → `models.Index(fields=['name', 'category'])`

4. **Avoid ordering/filtering on non-indexed large text fields** → slows DB

8️⃣ Summary Flow

1. Model field indexing → database speedup
2. Queryset ordering/filtering on indexed field → fast API
3. DRF Filter/Search/Ordering → flexible API
4. Pagination → memory & network optimization

- **Database indexing** (indexed fields for fast queries)
- **Optimized queryset** (select_related, only, prefetch_related)
- **DRF search & ordering** (filtering, ordering on indexed fields)
- **Pagination** (LimitOffsetPagination)
- Ready-to-use **API endpoints**

1️⃣ Project Structure

```
drf_indexing_project/
├─ drf_indexing_project/
│   ├─ settings.py
│   ├─ urls.py
│   └─ wsgi.py
├─ products/
│   ├─ models.py
│   ├─ serializers.py
│   ├─ views.py
│   ├─ urls.py
│   └─ admin.py
├─ manage.py
└─ requirements.txt
```

2️⃣ requirements.txt

```
Django>=4.2
djangorestframework>=3.14
```

```
django-filter>=23.2
```

3️⃣ products/models.py

```
from django.db import models

class Category(models.Model):
    name = models.CharField(max_length=50, unique=True)

    def __str__(self):
        return self.name

class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    category = models.ForeignKey(Category, on_delete=models.CASCADE,
related_name='products')
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        indexes = [
            models.Index(fields=['name']),
            models.Index(fields=['category']),
            models.Index(fields=['-created_at']),
        ]
        ordering = ['created_at']

    def __str__(self):
        return self.name
```

4️⃣ products/serializers.py

```
from rest_framework import serializers
from .models import Product, Category

class CategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = Category
        fields = ['id','name']

class ProductSerializer(serializers.ModelSerializer):
    category = CategorySerializer(read_only=True)
```

```
    class Meta:
        model = Product
        fields = ['id','name','price','category','created_at']
```

5️⃣ products/views.py

```
from rest_framework import viewsets, filters
from .models import Product
from .serializers import ProductSerializer
from django_filters.rest_framework import DjangoFilterBackend

class ProductViewSet(viewsets.ModelViewSet):
    queryset = Product.objects.select_related('category').all()  #
optimized queryset
    serializer_class = ProductSerializer

    filter_backends = [DjangoFilterBackend, filters.OrderingFilter,
filters.SearchFilter]
    filterset_fields = ['category__name', 'price']  # filtering only on
indexed fields
    search_fields = ['name']                        # indexed search
    ordering_fields = ['created_at', 'price']       # only indexed fields
recommended
    ordering = ['created_at']                       # default first to
last
```

6️⃣ products/urls.py

```
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import ProductViewSet

router = DefaultRouter()
router.register(r'products', ProductViewSet, basename='products')

urlpatterns = [
    path('', include(router.urls)),
]
```

7️⃣ drf_indexing_project/urls.py

```python
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('products.urls')),  # API root
]
```

8️⃣ settings.py (DRF & Pagination)

```python
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'django_filters',
    'products',
]

REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS':
'rest_framework.pagination.LimitOffsetPagination',
    'PAGE_SIZE': 20,
    'DEFAULT_FILTER_BACKENDS': [
        'django_filters.rest_framework.DjangoFilterBackend',
        'rest_framework.filters.OrderingFilter',
        'rest_framework.filters.SearchFilter',
    ]
}
```

✅ 9️⃣ Example API Calls

1. **Get first 20 products** (first-to-last by created_at)

```
GET /api/products/
```

2. **Pagination**

```
GET /api/products/?limit=10&offset=20
```

3. **Ordering by price descending**

```
GET /api/products/?ordering=-price
```

4. **Search by name**

```
GET /api/products/?search=phone
```

5. **Filter by category**

```
GET /api/products/?category__name=Electronics
```